МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
02.03.01 «Математика и компьютерные науки»

Отчёт о выполнении курсовой работы

по дисциплине «Структуры данных»

Разработка игрового бота для игры «Крестики-нолики»

Выполнила: студентка группы
5130201/50301

Переверзева М.А.

Проверил: ассистент

Кирпиченко С.Р.

Санкт-Петербург, 2026

Содержание

Введение **3**

1 Постановка задачи **4**

2 Математическое описание **5**

2.1 Модель игры 5

2.2 Оценка позиции 6

2.3 Отбор ходов-кандидатов 6

2.4 Минимакс (негамакс) 7

2.5 Альфа-бета отсечение 8

2.6 Итеративное увеличение глубины и ограничение по времени 9

2.7 Порядок работы алгоритма 9

3 Особенности реализации **11**

3.1 Программный интерфейс 11

3.2 Внутреннее представление поля 11

3.3 Инкрементальный пересчёт оценки 12

3.4 Генерация кандидатов и форсированные ходы 13

3.5 Поиск и управление временем 14

3.6 Используемые структуры данных и случайность 15

3.7 Организация разработки и непрерывная интеграция 15

4 Результаты работы программы **18**

4.1 Методика тестирования 18

4.2 Результаты автоматических тестов 19

4.3 Результаты партий против базового уровня 19

4.4 Время расчёта хода 20

Заключение **22**

Список литературы **23**

Приложение А. Интерфейс игрового бота **24**

Приложение Б. Реализация игрового бота **27**

Введение

Игра «крестики-нолики» — классический пример дискретной игры с полной информацией, позволяющий изучать методы поиска в пространстве состояний: минимакс, альфа-бета отсечение и эвристическую оценку позиции [1]. На поле 20×20 с препятствиями и условием победы «пять в ряд» полный перебор практически невозможен, поэтому необходимы эффективные алгоритмы выбора хода с ограничением времени вычисления.

Цель курсовой работы — разработать игрового бота для данного варианта игры, обеспечивающего корректность ходов и ограничение времени вычисления. Для этого позицию оценивают приблизительно, отбрасывают заведомо плохие варианты и сокращают перебор возможных ходов.

Бот встраивается в готовую игровую инфраструктуру, предоставленную в рамках курса «Структуры данных» [2]. Разработка велась с использованием GitHub Projects [8], отдельных веток и Pull Request. Для проверки работы бота проведены автоматические тесты и серии игр против базовых игроков.

1 Постановка задачи

Для достижения цели необходимо решить следующие задачи:

1. Реализовать выбор допустимого хода с учётом занятых клеток, препятствий и границ поля.
2. Разработать оценочную функцию позиции, учитывающую перспективные линии и угрозы соперника.
3. Реализовать поиск лучшего хода с альфа-бета отсечением и сокращением числа рассматриваемых кандидатов.
4. Организовать выбор хода с ограничением времени вычисления не более 100 мс.
5. Оптимизировать обработку игрового поля и пересчёт оценки позиции для ускорения выбора хода.
6. Оценить работу бота по результатам партий против базовых игроков и измерениям времени вычисления хода.

Результатом курсовой работы должна стать программная реализация игрового бота, который получает текущее состояние игрового поля и возвращает координаты выбранного хода. Бот должен учитывать расположение препятствий и фигур игроков, выбирать допустимые ходы на основе разработанного алгоритма и выполнять вычисления в пределах установленного ограничения по времени.

2 Математическое описание

2.1 Модель игры

Поле — прямоугольная таблица размером 20 строк на 20 столбцов. Каждая клетка поля находится в одном из четырёх состояний: свободна, содержит препятствие, содержит знак первого игрока (X) или знак второго игрока (O). Игроки ходят по очереди в свободную клетку, начинает первый игрок. Партия завершается, когда какой-либо игрок составил линию из пяти или более своих знаков подряд по горизонтали, вертикали или одной из двух диагоналей, либо когда все клетки поля заняты.

Особое правило: если линию первым составил второй игрок, он побеждает немедленно. Если линию первым составил первый игрок, второй игрок получает право на последний ход; если он тоже успевает составить линию, объявляется ничья, иначе побеждает первый игрок.

Окном будем называть любые пять клеток поля, идущих подряд вдоль строки, столбца или диагонали (под 45°). Составить выигрышную линию можно только внутри окна, свободного от препятствий и не содержащего знаков соперника.

Для клетки (x, y) множество всех окон длины пять, проходящих через неё, задаётся формулой

$$\mathcal{W}(x,y) = \left\{ \left\{ \big(x + (i-k)d_x,\ y + (i-k)d_y\big) \right\}_{i=0}^{4} \,\middle|\, \begin{array}{c} (d_x, d_y) \in D, \\ k \in \{0,1,2,3,4\}, \\ \text{все клетки} \\ \text{принадлежат полю} \end{array} \right\}, \tag{1}$$

где

$$D = \{(1,0), (0,1), (1,1), (1,-1)\}.$$

Множество $\mathcal{W}(x, y)$ содержит все окна длины пять, проходящие через клетку (x, y). Оно используется для инкрементального пересчёта оценки после выполнения или отмены хода.

2.2 Оценка позиции

Пусть E — число, характеризующее позицию с точки зрения бота: чем больше E, тем позиция выгоднее для бота, чем меньше (отрицательнее) — тем выгоднее для соперника.

Величина E равна сумме вкладов $v(W)$ по всем окнам W поля:

$$E = \sum_{W} v(W). \tag{2}$$

Вклад одного окна определяется так:

$$v(W) = \begin{cases} 0, & \text{если в окне есть препятствие, знаки обоих игроков или нет знаков;} \\ +w(k), & \text{если в окне } k \text{ знаков бота и нет знаков соперника;} \\ -w(k), & \text{если в окне } k \text{ знаков соперника и нет знаков бота.} \end{cases} \tag{3}$$

Здесь k — число знаков в окне, а весовая функция $w(k)$ задана в таблице 1.

Таблица 1 — Весовая функция окна $w(k)$

k (число одинаковых знаков в окне)	0	1	2	3	4	5
$w(k)$	0	1	10	300	20 000	10 000 000

Значения $w(k)$ подобраны так, чтобы каждая дополнительная фигура в окне резко увеличивала вклад: наличие четырёх своих знаков в свободном окне (готовая угроза выигрыша следующим ходом) весит на четыре порядка больше, чем одиночный знак, а собранная линия из пяти оценивается как безусловная победа.

2.3 Отбор ходов-кандидатов

Перебор всех свободных клеток поля на каждом уровне поиска приводит к неприемлемому времени работы, поэтому рассматриваются только клетки-кандидаты: свободные клетки, отстоящие не более чем на 2 клетки (по каждой координате) от какой-либо уже занятой клетки. Если поле полностью пустое (первый ход партии), кандидатом считается единственная клетка — ближайшая к центру поля.

Каждому кандидату x присваивается приоритет — оценка немедленной полезности хода в эту клетку для игрока, который сейчас ходит (назо-

вём его «текущим»). Приоритет вычисляется как сумма вкладов по всем окнам, содержащим клетку x и свободным от препятствий:

- пустое окно (в нём ещё нет ни одного знака) даёт вклад $+1$;
- окно, где после гипотетического хода будет k знаков текущего игрока и ни одного знака соперника, даёт вклад $a(k)$ — «атакующая» ценность;
- окно, где есть k знаков соперника и ни одного знака текущего игрока, даёт вклад $d(k)$ — «защитная» ценность, отражающая необходимость блокировать угрозу соперника.

Коэффициенты $a(k)$ и $d(k)$ приведены в таблице 2.

Таблица 2 — Весовые коэффициенты приоритета клетки

k	1	2	3	4
$a(k)$ — свои знаки	10	100	1 000	10 000 000
$d(k)$ — чужие знаки	5	50	500	1 000 000

Кандидаты сортируются по убыванию приоритета; в дальнейшем переборе участвуют только 12 лучших (на первом уровне перебора — 30 лучших, поскольку там цена ошибки выше).

Форсированный ход. Если приоритет лучшего кандидата не меньше 1 000 000, множество рассматриваемых ходов сокращается до одного кандидата. Такой высокий приоритет соответствует сильной тактической возможности или угрозе. Это эвристическое сокращение перебора ускоряет поиск, но само по себе не гарантирует, что выбранный ход оптимален во всех игровых ситуациях, в частности при наличии нескольких одновременных угроз.

2.4 Минимакс (негамакс)

Значение позиции для игрока, который сейчас должен ходить, определяется рекурсивно (алгоритм минимакс в форме негамакс [1]: оценка всегда берётся с точки зрения ходящего, поэтому знак значения меняется на противоположный при переходе хода к сопернику):

- если ход завершает линию из пяти и более своих знаков — значение хода равно очень большому числу (100 000 000) минус номер полухода партии: чем быстрее достигается выигрыш, тем выше значение;

– если достигнута заданная глубина перебора D полуходов и партия не завершена — значением листа считается E, если сейчас ходит бот, и $-E$, если ходит соперник;
– иначе значение хода равно минус значению позиции для соперника, который делает следующий ход из получившегося состояния (рекурсивный вызов той же процедуры с противоположным знаком).

Значением позиции для игрока считается максимум значений всех его ходов-кандидатов.

Для нетерминального состояния при положительной глубине поиска рекуррентное соотношение Negamax имеет вид

$$V(s, d) = \max_{a \in A(s)} \big[-V(T(s, a), d - 1)\big], \tag{4}$$

где s — текущее состояние игры, d — оставшаяся глубина поиска, $A(s)$ — множество рассматриваемых ходов, $T(s, a)$ — состояние после выполнения хода a. При достижении нулевой глубины используется эвристическая оценка с точки зрения игрока, которому принадлежит ход.

2.5 Альфа-бета отсечение

Отсечение (альфа-бета). Пусть при переборе ходов игрока уже найден ход со значением α (нижняя граница того, что игрок гарантированно может получить). При разборе очередного хода, как только у соперника находится ответ, после которого итоговое значение для игрока становится не выше α, дальнейшие ответы соперника на этот ход можно не проверять: этот ход заведомо не лучше уже найденного, и разбор оставшихся ответов не может изменить общий результат перебора. Это позволяет отбрасывать значительную часть дерева перебора без потери точности итогового результата.

После оценки очередного хода со значением v нижняя граница обновляется по правилу

$$\alpha \leftarrow \max(\alpha, v). \tag{5}$$

Если выполняется условие

$$\alpha \geq \beta, \tag{6}$$

оставшиеся ходы данного узла не рассматриваются. При переходе к ходу соперника знак оценки меняется, поэтому рекурсивный вызов Negamax выполняется с границами $(-\beta, -\alpha)$. При достижении $\alpha \geq \beta$ оставшиеся ходы не могут улучшить решение родительского узла в рамках рассматриваемого дерева поиска.

2.6 Итеративное увеличение глубины и ограничение по времени

Поскольку заранее не известно, какая глубина перебора уложится в отведённое время, применяется следующая схема:

- перебор выполняется последовательно для глубин $D = 2, 4, 6, \ldots$ полуходов (чётные глубины выбраны так, чтобы оценка листа всегда ставилась после ответного хода соперника, а не посреди обмена ходами);
- перед каждой новой глубиной лучший ход, найденный на предыдущей глубине, рассматривается первым — это ускоряет отсечения на следующей итерации;
- для каждого хода отслеживается прошедшее время; как только оно превышает установленный лимит (по умолчанию 70 мс), текущая, ещё не завершённая глубина прекращается, и используется лучший ход последней полностью завершённой глубины;
- если на какой-либо глубине найден гарантированный выигрыш или неизбежный проигрыш, дальнейшее увеличение глубины прекращается, так как результат уже не может измениться.

Итоговый ход — координаты клетки, выбранной на последней полностью завершённой глубине перебора. Если поиск не завершил ни одной глубины, используется запасной ход, выбранный по локальному приоритету.

2.7 Порядок работы алгоритма

Последовательность выбора хода в реализованном алгоритме:

1. Прочитать игровое состояние и вычислить вспомогательные данные для оценки позиции.
2. Сформировать и отсортировать кандидатов, учитывая атакующие и защитные угрозы.
3. Запомнить лучший по локальному приоритету допустимый ход

как запасной вариант.

4. Запустить поиск негамакс с альфа-бета отсечением последовательно на глубинах $2, 4, 6, \ldots$.
5. При исчерпании времени отбросить незавершённую итерацию.
6. Вернуть лучший ход последней завершённой итерации либо запасной вариант.

3 Особенности реализации

3.1 Программный интерфейс

Алгоритм реализован в виде класса `MyPlayer` (файлы `src/player/my_player.hpp`, `src/player/my_player.cpp`), реализующего интерфейс игры `ttt::game::IPlayer`. Публичный интерфейс класса приведён в листинге 1.

Листинг 1 — Публичный интерфейс класса `MyPlayer`

```
class MyPlayer : public IPlayer {
 public:
 MyPlayer(const char *name) : m_sign(Sign::NONE), m_name(name) {}
 void set_sign(Sign sign) override;
 Point make_move(const State &game) override;
 const char *get_name() const override;

 // Настройка и статистика (для тестов и отчета).
 void set_time_limit_ms(int ms) { m_time_limit_ms = ms; }
 int last_depth() { return m_last_depth; }
 ...
};
```

Игра вызывает `set_sign(Sign)` один раз в начале партии, сообщая, каким знаком играет бот, и `make_move(constState&)` перед каждым ходом бота, ожидая в ответ координаты свободной клетки. Дополнительные методы (`set_time_limit_ms`, `last_depth`, `last_nodes`, `last_time_ms`) предназначены для настройки лимита времени и получения статистики поиска в автоматических тестах и при измерении производительности.

3.2 Внутреннее представление поля

Состояние `State`, полученное от игры, копируется в собственный массив `std::vector<std::int8_t>` размером rows $\times$ cols (метод `load`), где каждой клетке соответствует одно из четырёх значений: свободна, знак бота, знак соперника, препятствие. Внутреннее представление поля использует один

байт на клетку и не предназначено для уменьшения объёма памяти относительно исходного `State`, в котором состояния клеток кодируются двумя битами. Его основное преимущество состоит в возможности изменять и восстанавливать игровую позицию непосредственно в одном массиве при рекурсивном поиске, без создания полной копии состояния для каждого рассматриваемого хода.

Дополнительно хранится вспомогательный массив `m_near` того же размера, в котором для каждой клетки накапливается число занятых клеток в радиусе 2 вокруг неё (метод `bump_near`). Ненулевое значение в этом массиве — признак того, что клетка является кандидатом на ход; это позволяет перечислять кандидатов за один проход по полю без вычисления расстояний в момент перебора.

3.3 Инкрементальный пересчёт оценки

Оценка позиции E (формула (2)) при полной перезагрузке поля считается суммированием вкладов всех окон; листинг 2 показывает вычисление вклада одного окна.

Листинг 2 — Вычисление вклада одного окна в оценку позиции

```
long long MyPlayer::window_value(int sx, int sy, int d) const {
 int me = 0, opp = 0;
 for (int i = 0; i < m_len; ++i) {
  std::int8_t c = at(sx + DX[d] * i, sy + DY[d] * i);
  if (c == WALL)
  return 0;
  if (c == ME)
  ++me;
  else if (c == OPP)
  ++opp;
 }
 if (me && opp)
 return 0;
 if (me)
 return m_W[me];
 if (opp)
```

```
 return -m_W[opp];
 return 0;
}
```

Пересчитывать сумму по всем окнам поля после каждого пробного хода в переборе было бы слишком медленно. Поэтому при постановке или снятии знака в клетке (методы `apply`/`undo`) из общей суммы вычитаются старые вклады только тех окон, что проходят через изменённую клетку (таких окон не более $4 \times 5 = 20$ — по четыре направления и по пять положений окна), а затем добавляются новые вклады тех же окон. Такой инкрементальный пересчёт занимает время, не зависящее от размера поля, и является основным фактором, позволяющим уложиться в отведённый лимит времени на ход при переборе на глубину в несколько полуходов.

3.4 Генерация кандидатов и форсированные ходы

Функция `gen` (листинг 3) перечисляет клетки-кандидаты (те, где `m_near` > 0), присваивает им приоритет через вспомогательную функцию `cell_priority` (реализующую формулу из раздела 2.3) и оставляет только `beam` лучших по приоритету, используя частичную сортировку `std::partial_sort` со сложностью $O(N \log \mathrm{beam})$ [3, 4]. Если приоритет лучшего кандидата не ниже порога, соответствующего форсированному ходу, список кандидатов сокращается до одного элемента.

Листинг 3 — Генерация и отбор кандидатов на ход

```
int MyPlayer::gen(int ply, std::int8_t mover, int beam) {
 auto &v = m_cands[ply];
 v.clear();
 for (int x = 0; x < m_nx; ++x)
 for (int y = 0; y < m_ny; ++y) {
  int idx = x * m_ny + y;
  if (m_b[idx] == EMPTY && m_near[idx] > 0)
  v.push_back({x, y, cell_priority(x, y, mover)});
 }
 int n = std::min<int>(beam, (int)v.size());
 std::partial_sort(v.begin(), v.begin() + n, v.end(),
```

```
 [](const Cand &a, const Cand &b) { return a.pr > b.pr; });
 // Форсированные ходы: есть ход, собирающий линию, или надо блокиро
 // соперника, у которого до линии не хватает одной клетки.
 if (n > 0 && v[0].pr >= m_def[m_len - 1])
 n = 1;
 return n;
}
```

Листинг 4 — Фрагмент рекурсивного поиска и отсечения

```
if (val > best)
best = val;
if (best > alpha)
alpha = best;
if (alpha >= beta)
break;
```

3.5 Поиск и управление временем

Собственно перебор реализован методом `negamax` (рекурсивный обход дерева ходов с альфа-бета отсечением, раздел 2.4) и методом `search_root` (отдельная реализация верхнего уровня перебора, которая, в отличие от внутренних узлов, запоминает не только оценку, но и сами координаты лучшего хода). Итеративное углубление и контроль времени реализованы в `make_move`: перед каждым вызовом запоминается момент начала расчёта (`std::chrono::steady_clock` [4]), вычисляется дедлайн, и внутри `negamax` раз в 256 просмотренных узлов проверяется, не истекло ли отведённое время; при превышении лимита текущая ветвь перебора прерывается флагом `m_timeout`, и результат текущей (незавершённой) глубины отбрасывается. Начальным запасным ходом, который используется, если не завершилась ни одна полная глубина перебора (например, на очень перегруженном препятствиями поле), служит кандидат с наибольшим приоритетом из `gen` — то есть локально лучший ход по одноходовой эвристике.

Листинг 5 — Фрагмент итеративного углубления в make_move

```
for (int depth = 2; depth <= MAX_DEPTH; depth += 2) {
 int rx, ry;
 int val = search_root(depth, bx, by, rx, ry);
 if (m_timeout || rx < 0)
 break;
 bx = rx;
 by = ry;
 m_last_depth = depth;
 if (std::abs(val) >= WIN - 100)
 break;
}
```

3.6 Используемые структуры данных и случайность

Генератор случайных чисел в алгоритме не используется. Однако поиск ограничен временем, поэтому при одинаковом состоянии поля из-за различий в скорости выполнения программы может быть достигнута разная глубина перебора и, как следствие, выбран другой ход. Из структур данных, помимо описанных выше плоских массивов на основе `std::vector`, используется небольшой массив `std::vector<Cand>` на каждый уровень перебора (переиспользуемый между вызовами, чтобы не выделять память многократно), где `Cand` — простая структура «координаты + приоритет». Все веса (`m_W`, `m_att`, `m_def`) — константы, зависящие только от длины выигрышной линии (`win_len`) и вычисляемые один раз при загрузке состояния (метод `load`), что позволяет использовать один и тот же класс без изменений для других значений длины линии, если параметры игры изменятся.

3.7 Организация разработки и непрерывная интеграция

Разработка велась по задачам (Issues) на Kanban-доске GitHub Projects [8] с тремя колонками (Backlog, In Progress, Done). Каждая задача реализована в отдельной ветке репозитория (`feature/#N_...`), коммиты помечены номером задачи (например, `[#3] ImplementMyPlayer...`), а слияние с веткой `dev` выполнено через Pull Request с пометкой `Closes#N`, закрывающей соответствующий Issue.

История веток и организация задач показаны на рисунках 1 и 2.

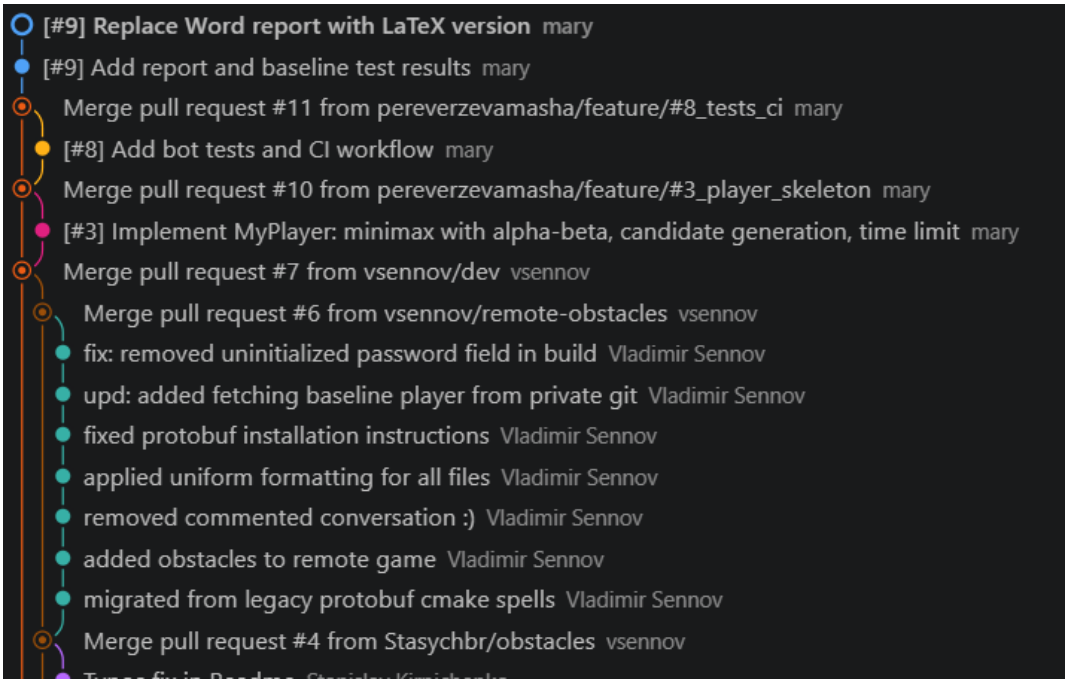


Рис. 1 — Граф веток и запросов на слияние собственного проекта

Для автоматической проверки работоспособности проекта настроен процесс непрерывной интеграции на основе GitHub Actions [7] (файл `.github/workflows/ci.yml`, листинг 6). При каждом push в ветки `main`, `dev` и `feature/**` и при открытии Pull Request в `main` или `dev` запускается сборка проекта в конфигурации Release и полный прогон тестов через CTest.

Листинг 6 — Конфигурация непрерывной интеграции (`.github/workflows/ci.yml`)

```
# Кладётся в .github/workflows/ci.yml
# Собирает проект (без предкомпилированной библиотеки baseline, так
# закрытая и недоступна CI) и запускает CTest.
name: CI

on:
push:
branches: [ main, dev, "feature/**" ]
pull_request:
branches: [ main, dev ]

jobs:
```

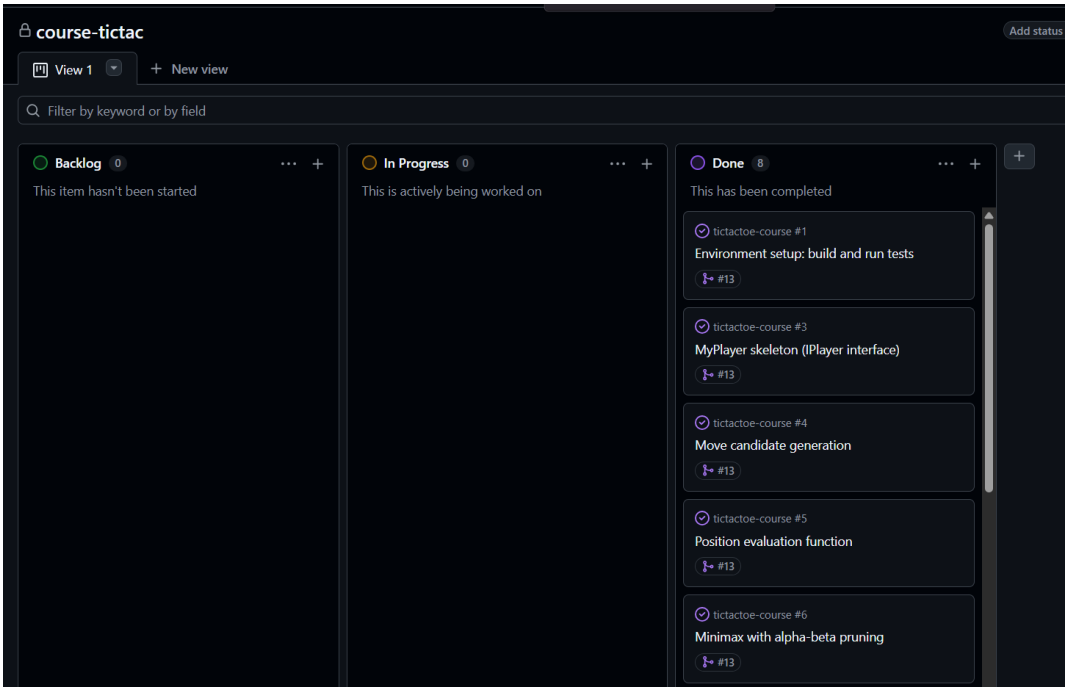


Рис. 2 — Kanban-доска GitHub Projects

```
build-and-test:
runs-on: ubuntu-latest
steps:
- name: Checkout
uses: actions/checkout@v4

- name: Configure
run: |
mkdir -p build
cd build
cmake .. -DCMAKE_BUILD_TYPE=Release

- name: Build
run: cmake --build build --config Release -j

- name: Test
run: |
cd build
ctest --output-on-failure
```

4 Результаты работы программы

4.1 Методика тестирования

Все тесты проводились при фиксированных параметрах партии, установленных заданием: поле 20×20, длина выигрышной линии 5, `playable_part` = 0,75, `gap` = 1, `max_obstacle_len` = 50. Лимит времени на ход бота — 70 мс (метод `set_time_limit_ms`), что оставляет запас до предельных 100 мс с учётом накладных расходов на копирование состояния.

Предоставленная библиотека игроков базового уровня (`libtttcore.a`) собрана под Linux (формат ELF) и не может быть подключена к проекту, собранному компилятором MSVC в Windows. Поэтому итоговые испытания выполнены в среде WSL (Ubuntu) на компьютере с ОС Windows: компилятор GCC 15.2.0, CMake 4.2.3 [5], конфигурация сборки Release. Процессор: 13th Gen Intel(R) Core(TM) i7-13620H.

В проекте используются следующие автоматические тесты (папка `tests/`):

- `smoke_test_player` (`tests/test_my_player.cpp`) — базовая проверка работоспособности игрока, предоставленная шаблоном проекта;
- `test_player_stats` (`tests/test_stats.cpp`) — 100 партий бота против самого себя, проверка устойчивости бота;
- `test_my_bot` (`tests/test_my_bot.cpp`, добавлен в рамках данной работы) — три целевые проверки: корректность возвращаемых ходов (клетка внутри поля и свободна), полная партия бота против самого себя без дисквалификаций, время одного хода не более 100 мс;
- `test_with_baseline` (`tests/test_baseline.cpp`) — одна партия против игрока базового уровня с выводом поля после каждого хода, используется для наглядной проверки;
- `test_stats_vs_baseline` (`tests/test_stats_vs_baseline.cpp`) — серия из 100 партий против игрока базового уровня с подсчётом побед, поражений, ничьих и времени хода.

Для воспроизведения результатов в WSL выполняются команды `cmake..-DBUILD_TTTCORE=PREBUILT-DCMAKE_BUILD_TYPE=Release` и `cmake--build.-j`, после чего запускается `./tests/test_stats_vs_`

baseline. Для испытания против сильного игрока в этом файле заменяется строка получения соперника: вместо ttt::baseline::get_easy_player используется ttt::baseline::get_harder_player, остальной код не меняется. Результаты сохранялись в текстовые файлы командой tee.

Сборка и запуск тестов автоматизированы через GitHub Actions (раздел 3.7): при каждом коммите в отслеживаемые ветки CI собирает проект в конфигурации Release и выполняет набор тестов командой ctest--output-on-failure. В CI предкомпилированная библиотека недоступна, поэтому тесты против baseline там не выполняются. Прогон CI на ветке dev завершился успешно (все доступные тесты пройдены, общее время около 24 минут).

4.2 Результаты автоматических тестов

Таблица 3 — Результаты прогона тестов CTest

Тест	Что проверяет	Результат
smoke_test_player	Базовая работоспособность игрока	Пройден
test_my_bot	Легальность хода, партия без дисквалификации, время хода ≤ 100 мс (3 проверки)	Пройден (3/3)
test_player_stats	100 партий бота против самого себя	Пройден (в CI)
test_with_baseline	Одна партия против сильного игрока базового уровня	Завершена, ничья

4.3 Результаты партий против базового уровня

Серия из 100 партий против каждого из игроков базового уровня сыграна тестом test_stats_vs_baseline. Бот играл за первого игрока (X). Процент побед без учёта ничьих вычислялся как отношение числа побед бота к сумме побед и поражений бота. Результаты приведены в таблице 4.

Таблица 4 — Результаты серии из 100 партий против игроков базового уровня

Соперник	Побед	Пораж.	Ничьих	Ошибок	% побед без ничьих
Базовый («лёгкий»)	94	4	2	0	95,9
Повышенный («сложный»)	25	2	73	0	92,6

Требование задания (более 50% побед над игроком базового уровня и более 25% над игроком повышенного уровня без учёта ничьих) выполнено с запасом: $94/(94+4) \approx 95{,}9\%$ и $25/(25+2) \approx 92{,}6\%$ соответственно. Ошибок и дисквалификаций ни в одной из 200 партий не было.

Против сильного соперника подавляющее большинство партий (73 из 100) заканчивается ничьей: оба игрока хорошо блокируют угрозы, и поле заполняется без выигрышной линии. Ничья не является поражением бота, поэтому результат полезно оценить и с учётом ничьих. Против лёгкого игрока бот не проиграл в $94+2=96$ партиях из 100 (96%), против сильного — в $25+73=98$ партиях из 100 (98%). Доля побед от общего числа партий составила 94% и 25% соответственно. Критерий задания сформулирован без учёта ничьих, поэтому формальным показателем остаётся значение из последнего столбца таблицы 4.

```
Testing MyPlayer vs baseline easy player
MyPlayer wins: 94
BaselineEasy wins: 4
draws:  2
errors: 0

MyPlayer play time:
 - move time (ms): 60.5072
 - event time (ms): 0.000213398

BaselineEasy play time:
 - move time (ms): 0.00201595
 - event time (ms): 0.00621737

game process average time: 30.2646 (ms)

real    2m43.241s
user    2m43.211s
sys     0m0.016s
```

Рис. 3 — Результаты партий против лёгкого базового игрока

```
Testing MyPlayer vs baseline easy player
MyPlayer wins: 25
BaselineHard wins: 2
draws:  73
errors: 0

MyPlayer play time:
 - move time (ms): 67.9159
 - event time (ms): 0.000197584

BaselineHard play time:
 - move time (ms): 0.00296645
 - event time (ms): 0.00352726

game process average time: 33.9664 (ms)

real    13m7.764s
user    13m7.674s
sys     0m0.051s
```

Рис. 4 — Результаты партий против сложного базового игрока

4.4 Время расчёта хода

Время каждого хода измеряется программно: тест `test_stats_vs_baseline` выводит среднее время хода каждого игрока в миллисекундах (таблица 5).

Среднее время хода бота (60,5 и 67,9 мс) укладывается в требуемые 50–100 мс. Оно зависит от лимита 70 мс: на ходах, где перебор завершается раньше (например, при форсированных ходах), расчёт идёт быстрее, поэтому среднее значение ниже лимита. Максимальное время хода и достигнутая глубина перебора в данных серийных тестах не фиксировались. Поэтому по приведённым средним значениям нельзя доказать, что каждый отдельный ход завершался не дольше 100 мс; для такой проверки требуется отдельно

Таблица 5 — Время хода бота и длительность серий

Характеристика	Против лёгкого	Против сильного
Среднее время хода бота, мс	60,5	67,9
Среднее время хода соперника, мс	0,002	0,003
Число партий в серии	100	100
Общее время серии	2 мин 43 с	13 мин 8 с

измерить и вывести максимум по всем ходам.

Заключение

В ходе курсовой работы реализован игровой бот для варианта «Крестики-нолики» на поле 20×20 с препятствиями и условием победы «пять в ряд». Основной результат работы — класс `MyPlayer`, который получает состояние игрового поля и возвращает координаты допустимого хода.

Поставленные задачи решались следующим образом: для выбора хода учитываются границы поля, занятые клетки и препятствия; для сравнения позиций применяется эвристическая оценка по отрезкам длины пять; поиск осуществляется с альфа-бета отсечением, отбором перспективных кандидатов и итеративным углублением. Инкрементальный пересчёт оценки уменьшает объём вычислений при рассмотрении очередного хода. Время поиска ограничивается программным таймером.

Работоспособность бота оценивалась по результатам испытаний в готовой инфраструктуре проекта. В 100 партиях против лёгкого базового игрока получены 94 победы, 4 поражения и 2 ничьи; против сложного — 25 побед, 2 поражения и 73 ничьи. Доля побед без учёта ничьих составила соответственно 95,9% и 92,6%, что превышает заданные пороги. Среднее время выбора хода составило 60,5 и 67,9 мс. Поскольку максимальное время по всем ходам в сериях не фиксировалось, эти данные подтверждают выполнение ограничения по среднему времени, но не гарантируют соблюдение порога в каждом отдельном ходе.

Объём работы: класс `MyPlayer` занимает около 320 строк в `my_player.cpp` и 72 строки в `my_player.hpp`, автоматические тесты — около 140 строк. Общие трудозатраты составили около 20 часов.

Таким образом, разработанный бот выбирает допустимые ходы и показывает результаты выше установленных требований к доле побед. Дальнейшее улучшение возможно за счёт более точного учёта открытых комбинаций, специальной обработки правила последнего хода второго игрока и кэширования уже рассмотренных позиций.

Список литературы

[1] Рассел С., Норвиг П. Искусственный интеллект: современный подход. 3-е изд. М.: Вильямс, 2018.

[2] vsennov. tictactoe-course: материалы курсовой работы по программированию на C++ [Электронный ресурс] // GitHub. URL: https://github.com/vsennov/tictactoe-course (дата обращения: 25.10.2026).

[3] Cormen T. H., Leiserson C. E., Rivest R. L., Stein C. Introduction to Algorithms. 3rd ed. Cambridge, MA: MIT Press, 2009.

[4] C++ reference [Электронный ресурс] // cppreference.com. URL: https://en.cppreference.com/ (дата обращения: 25.10.2026).

[5] CMake Reference Documentation [Электронный ресурс]. URL: https://cmake.org/cmake/help/latest/ (дата обращения: 25.10.2026).

[6] ctest(1) — CMake Documentation [Электронный ресурс]. URL: https://cmake.org/cmake/help/latest/manual/ctest.1.html (дата обращения: 25.10.2026).

[7] GitHub Actions documentation [Электронный ресурс] // GitHub Docs. URL: https://docs.github.com/actions (дата обращения: 25.10.2026).

[8] Planning and tracking with Projects [Электронный ресурс] // GitHub Docs. URL: https://docs.github.com/issues/planning-and-tracking-with-projects (дата обращения: 25.10.2026).

Приложение А. Интерфейс игрового бота

А.1. Файл src/player/my_player.hpp

```
#pragma once

#include "core/game.hpp"

#include <chrono>
#include <cstdint>
#include <vector>

namespace ttt::my_player {

 using game::Event;
 using game::IPlayer;
 using game::Point;
 using game::Sign;
 using game::State;

 class MyPlayer : public IPlayer {
  public:
  MyPlayer(const char *name) : m_sign(Sign::NONE),
  m_name(name) {}
  void set_sign(Sign sign) override;
  Point make_move(const State &game) override;
  const char *get_name() const override;

  void set_time_limit_ms(int ms) { m_time_limit_ms = ms; }
  int last_depth() const { return m_last_depth; }
  long long last_nodes() const { return m_last_nodes; }
  double last_time_ms() const { return m_last_ms; }

  private:
  struct Cand {
```

```
 int x;
 int y;
 long long pr;
};

Sign m_sign;
const char *m_name;

int m_nx = 0, m_ny = 0, m_len = 5;
std::vector<std::int8_t> m_b;
std::vector<int> m_near;
long long m_score = 0;
long long m_W[16], m_att[16], m_def[16];
std::vector<std::vector<Cand>> m_cands;

int m_time_limit_ms = 70;
std::chrono::steady_clock::time_point m_deadline;
bool m_timeout = false;
long long m_nodes = 0;
int m_last_depth = 0;
long long m_last_nodes = 0;
double m_last_ms = 0;

void load(const State &game, Sign me);
std::int8_t at(int x, int y) const;
long long window_value(int sx, int sy, int d) const;
void apply(int x, int y, std::int8_t p);
void undo(int x, int y, long long saved);
void bump_near(int x, int y, int delta);
bool wins(int x, int y, std::int8_t p) const;
long long cell_priority(int x, int y,
std::int8_t mover) const;
int gen(int ply, std::int8_t mover, int beam);
int negamax(int depth, int ply, int alpha,
```

```
  int beta, std::int8_t side);
  int search_root(int depth, int hint_x, int hint_y,
  int &bx, int &by);
 };

};
```

Приложение Б. Реализация игрового бота

Б.1. Файл src/player/my_player.cpp

```
#include "player/my_player.hpp"

#include <algorithm>
#include <cstdlib>

namespace ttt::my_player {

 namespace {
  constexpr std::int8_t EMPTY = 0, ME = 1, OPP = 2, WALL = 3;
  constexpr int DX[4] = {1, 0, 1, 1};
  constexpr int DY[4] = {0, 1, 1, -1};
  constexpr int INF = 1000000000;
  constexpr int WIN = 100000000;
  constexpr int ROOT_BEAM = 30;
  constexpr int BEAM = 12;
  constexpr int MAX_DEPTH = 16;

  long long pow10ll(int n) {
   long long r = 1;
   while (n-- > 0)
   r *= 10;
   return r;
  }
  std::int8_t other(std::int8_t s) { return s == ME ?
   OPP : ME; }
 }

 void MyPlayer::set_sign(Sign sign) { m_sign = sign; }

 const char *MyPlayer::get_name() const { return m_name; }
```

```
std::int8_t MyPlayer::at(int x, int y) const {
 if (x < 0 || y < 0 || x >= m_nx || y >= m_ny)
 return WALL;
 return m_b[x * m_ny + y];
}

void MyPlayer::load(const State &game, Sign me) {
 const auto &o = game.get_opts();
 m_nx = o.rows;
 m_ny = o.cols;
 m_len = std::min(std::max(o.win_len, 2), 15);
 m_b.assign(m_nx * m_ny, EMPTY);
 m_near.assign(m_nx * m_ny, 0);
 m_score = 0;
 m_cands.assign(MAX_DEPTH + 2, std::vector<Cand>());

 m_W[0] = m_att[0] = m_def[0] = 0;
 for (int c = 1; c < m_len; ++c) {
  m_W[c] = pow10ll(c - 1);
  m_att[c] = pow10ll(c);
  m_def[c] = pow10ll(c) / 2;
 }
 if (m_len >= 4) {
  m_W[m_len - 1] = 20000;
  m_W[m_len - 2] = 300;
 }
 m_W[m_len] = 10000000;
 m_att[m_len - 1] = 10000000;
 m_def[m_len - 1] = 1000000;

 for (int x = 0; x < m_nx; ++x)
 for (int y = 0; y < m_ny; ++y) {
  Sign s = game.get_value(x, y);
  std::int8_t v = EMPTY;
```

```
  if (s == Sign::WALL)
  v = WALL;
  else if (s == Sign::NONE)
  v = EMPTY;
  else
  v = (s == me) ? ME : OPP;
  m_b[x * m_ny + y] = v;
 }
 for (int x = 0; x < m_nx; ++x)
 for (int y = 0; y < m_ny; ++y)
 if (m_b[x * m_ny + y] == ME || m_b[x * m_ny + y] == OPP)
 bump_near(x, y, +1);

 for (int x = 0; x < m_nx; ++x)
 for (int y = 0; y < m_ny; ++y)
 for (int d = 0; d < 4; ++d)
 m_score += window_value(x, y, d);
}

long long MyPlayer::window_value(int sx, int sy,
int d) const {
 int me = 0, opp = 0;
 for (int i = 0; i < m_len; ++i) {
  std::int8_t c = at(sx + DX[d] * i, sy + DY[d] * i);
  if (c == WALL)
  return 0;
  if (c == ME)
  ++me;
  else if (c == OPP)
  ++opp;
 }
 if (me && opp)
 return 0;
 if (me)
```

```
 return m_W[me];
 if (opp)
 return -m_W[opp];
 return 0;
}

void MyPlayer::bump_near(int x, int y, int delta) {
 for (int dx = -2; dx <= 2; ++dx)
 for (int dy = -2; dy <= 2; ++dy) {
  int nx = x + dx, ny = y + dy;
  if (nx >= 0 && ny >= 0 && nx < m_nx && ny < m_ny)
  m_near[nx * m_ny + ny] += delta;
 }
}

void MyPlayer::apply(int x, int y, std::int8_t p) {
 for (int d = 0; d < 4; ++d)
 for (int k = 0; k < m_len; ++k)
 m_score -= window_value(x - DX[d] * k, y - DY[d] * k, d);
 m_b[x * m_ny + y] = p;
 for (int d = 0; d < 4; ++d)
 for (int k = 0; k < m_len; ++k)
 m_score += window_value(x - DX[d] * k, y - DY[d] * k, d);
 bump_near(x, y, +1);
}

void MyPlayer::undo(int x, int y, long long saved) {
 m_b[x * m_ny + y] = EMPTY;
 bump_near(x, y, -1);
 m_score = saved;
}

bool MyPlayer::wins(int x, int y, std::int8_t p) const {
 for (int d = 0; d < 4; ++d) {
```

```
  int cnt = 1;
  for (int s = 1; at(x + DX[d] * s, y + DY[d] * s)
  == p; ++s)
  ++cnt;
  for (int s = 1; at(x - DX[d] * s, y - DY[d] * s)
  == p; ++s)
  ++cnt;
  if (cnt >= m_len)
  return true;
 }
 return false;
}

long long MyPlayer::cell_priority(int x, int y,
std::int8_t mover) const {
 long long s = 0;
 for (int d = 0; d < 4; ++d)
 for (int k = 0; k < m_len; ++k) {
  int sx = x - DX[d] * k, sy = y - DY[d] * k;
  int mv = 0, ot = 0;
  bool bad = false;
  for (int i = 0; i < m_len; ++i) {
   std::int8_t c = at(sx + DX[d] * i, sy + DY[d] * i);
   if (c == WALL) {
    bad = true;
    break;
   }
   if (c == mover)
   ++mv;
   else if (c != EMPTY)
   ++ot;
  }
  if (bad || (mv && ot))
  continue;
```

```
  if (!mv && !ot)
  s += 1;
  else if (!ot)
  s += m_att[mv];
  else
  s += m_def[ot];
 }
 return s;
}

int MyPlayer::gen(int ply, std::int8_t mover, int beam) {
 auto &v = m_cands[ply];
 v.clear();
 for (int x = 0; x < m_nx; ++x)
 for (int y = 0; y < m_ny; ++y) {
  int idx = x * m_ny + y;
  if (m_b[idx] == EMPTY && m_near[idx] > 0)
  v.push_back({x, y, cell_priority(x, y, mover)});
 }
 int n = std::min<int>(beam, (int)v.size());
 std::partial_sort(v.begin(), v.begin() + n, v.end(),
 [](const Cand &a, const Cand &b) { return a.pr > b.pr; });
 if (n > 0 && v[0].pr >= m_def[m_len - 1])
 n = 1;
 return n;
}

int MyPlayer::negamax(int depth, int ply, int alpha,
int beta, std::int8_t side) {
 if ((++m_nodes & 255) == 0 &&
 std::chrono::steady_clock::now() >= m_deadline)
 m_timeout = true;
 if (m_timeout)
 return 0;
```

```
 int n = gen(ply, side, BEAM);
 if (n == 0)
 return 0;
 int best = -INF;
 for (int i = 0; i < n; ++i) {
  Cand c = m_cands[ply][i];
  long long saved = m_score;
  apply(c.x, c.y, side);
  int val;
  if (wins(c.x, c.y, side))
  val = WIN - ply;
  else if (depth <= 1)
  val = (int)(side == ME ? m_score : -m_score);
  else
  val = -negamax(depth - 1, ply + 1, -beta,
  -alpha, other(side));
  undo(c.x, c.y, saved);
  if (m_timeout)
  return 0;
  if (val > best)
  best = val;
  if (best > alpha)
  alpha = best;
  if (alpha >= beta)
  break;
 }
 return best;
}

int MyPlayer::search_root(int depth, int hint_x,
int hint_y, int &bx,
int &by) {
 int n = gen(0, ME, ROOT_BEAM);
 auto &v = m_cands[0];
```

```
 for (int i = 0; i < n; ++i)
 if (v[i].x == hint_x && v[i].y == hint_y) {
  std::rotate(v.begin(), v.begin() + i, v.begin() + i + 1);
  break;
 }
 int alpha = -INF, best = -INF;
 bx = by = -1;
 for (int i = 0; i < n; ++i) {
  Cand c = v[i];
  long long saved = m_score;
  apply(c.x, c.y, ME);
  int val;
  if (wins(c.x, c.y, ME))
  val = WIN;
  else
  val = -negamax(depth - 1, 1, -INF, -alpha, OPP);
  undo(c.x, c.y, saved);
  if (m_timeout)
  return 0;
  if (val > best) {
   best = val;
   bx = c.x;
   by = c.y;
  }
  if (best > alpha)
  alpha = best;
 }
 return best;
}

Point MyPlayer::make_move(const State &game) {
 auto t0 = std::chrono::steady_clock::now();
 load(game, game.get_current_player());
 m_deadline = t0 + std::chrono::milliseconds
```

```
(m_time_limit_ms);
m_timeout = false;
m_nodes = 0;
m_last_depth = 0;

int bx = -1, by = -1;
int n = gen(0, ME, ROOT_BEAM);
if (n > 0) {
 bx = m_cands[0][0].x;
 by = m_cands[0][0].y;
} else {
 int bd = INF;
 for (int x = 0; x < m_nx; ++x)
 for (int y = 0; y < m_ny; ++y)
 if (m_b[x * m_ny + y] == EMPTY) {
  int d = std::abs(2 * x - (m_nx - 1)) +
  std::abs(2 * y - (m_ny - 1));
  if (d < bd) {
   bd = d;
   bx = x;
   by = y;
  }
 }
}

if (n > 0) {
 for (int depth = 2; depth <= MAX_DEPTH; depth += 2) {
  int rx, ry;
  int val = search_root(depth, bx, by, rx, ry);
  if (m_timeout || rx < 0)
  break;
  bx = rx;
  by = ry;
  m_last_depth = depth;
```

```
    if (std::abs(val) >= WIN - 100)
    break;
   }
  }

  m_last_nodes = m_nodes;
  m_last_ms = std::chrono::duration<double, std::milli>(
  std::chrono::steady_clock::now() - t0)
  .count();
  if (bx < 0)
  return Point{0, 0};
  return Point{bx, by};
 }

};
```